Calentasker

API teszt dokumentáció

A vizsgaremeket készítették:

Fesető Imre 13.I

Kapus Kevin Kamill 13.D

Tartalom

1 Tesztesetek 3
1.1 Autentikáció és Felhasználói Folyamatok 3
1.1.1 Felhasználó regisztrációja 3
1.1.2 Fiók aktiválása 3
1.1.3 Felhasználó bejelentkezése 4
1.1.4 Fiók törlése 4
1.2 Csoportok és Együttműködés 6
1.2.1 Csoportok létrehozása 6
1.2.2 Csoporttag hozzáadása 6
1.2.3 Csoport törlése 7
1.3 Feladatkezelés 8
1.3.1 Személyes feladat hozzáadása 8
1.3.2 Csoportos feladat létrehozása 8
1.3.3 Feladat módosítása 9
1.3.4 Feladatok szűrése 10
1.3.5 Feladat törlése 11
1.4 Lekérések 12
1.4.1 Felhasználók lekérése 12
1.4.2 Adott felhasználó lekérése 12
1.4.3 Csoportok lekérése 12
1.4.4 Adott csoport lekérése 13
1.4.5 Feladatok lekérése 13
1.4.6 Adott feladat lekérése 14

1 Tesztesetek

1.1 Autentikáció és Felhasználói Folyamatok

1.1.1 Felhasználó regisztrációja

Teszteset ID	TC-USER-01
API végpont	/api/users/
HTTP metódus	POST
Elvárt működés / Ellenőrzés	A username, email, password, first_name és last_name megadása sikeresen létrehoz egy inaktív felhasználói fiókot.

```
def test_user_registration(self):
        """Test user registration process"""
        data = {
            'username': 'newuser',
            'email': 'newuser@example.com',
            'password': 'password123',
            'first_name': 'New',
            'last_name': 'User'
        }
        res = self.client.post(self.users_url, data)
        self.assertEqual(res.status_code, status.HTTP_201_CREATED)
        self.assertFalse(res.data['is_active'])
```

1.1.2 Fiók aktiválása

Teszteset ID	TC-USER-02
API végpont	/api/users/activate/
HTTP metódus	POST
Elvárt működés / Ellenőrzés	A regisztrációs e-mailben kapott uid és token átadása sikeresen aktiválja a felhasználót.

```
def test_user_activation(self):
        """Test token activation for a newly registered user"""
        # Create an inactive user manually
        new_user = User.objects.create_user(
            username='inactive_user',
            email='inactive@example.com',
            password='password123',
            is_active=False
        )
```

```
        # Simulate clicking email link
        uid = urlsafe_base64_encode(force_bytes(new_user.pk))
        token = default_token_generator.make_token(new_user)

        activate_res = self.client.post(f"{self.users_url}activate/", {
            'uid': uid,
            'token': token
        })
        self.assertEqual(activate_res.status_code, status.HTTP_200_OK)

        new_user.refresh_from_db()
        self.assertTrue(new_user.is_active)
```

1.1.3 Felhasználó bejelentkezése

Teszteset ID	TC-AUTH-01
API végpont	/api/login/
HTTP metódus	POST
Elvárt működés / Ellenőrzés	Érvényes hitelesítő adatok (pl. admin/admin) megadása esetén a rendszernek egy hitelesítési tokent kell visszaadnia.

```
def test_user_login(self):
        """Test user login returns a token and user details"""
        response = self.client.post(self.login_url, {
            'username': 'user1',
            'password': 'password123'
        })
        self.assertEqual(response.status_code, status.HTTP_200_OK)
        self.assertIn('token', response.data)
        self.assertEqual(response.data['username'], 'user1')
```

1.1.4 Fiók törlése

Teszteset ID	TC-DEL-01
API végpont	/api/users/{id}
HTTP metódus	DELETE
Elvárt működés / Ellenőrzés	Az id adatot megadva az id-hez rendelt felhasználót soft delete eljárás alapján törli (inaktívvá állítja)

```
def test_user_soft_deletion(self):
        """Test that deleting a user marks them as inactive"""
```

```
        self.client.force_authenticate(user=self.user2)

        # We delete user2 to test soft deletion
        res = self.client.delete(f"{self.users_url}{self.user2.id}/")
        self.assertEqual(res.status_code, status.HTTP_204_NO_CONTENT)

        # Retrieve from DB to check soft delete
        self.user2.refresh_from_db()
        self.assertFalse(self.user2.is_active)
```

1.2 Csoportok és Együttműködés

1.2.1 Csoportok létrehozása

Teszteset ID	TC-GRP-01
API végpont	/api/groups/
HTTP metódus	POST
Elvárt működés / Ellenőrzés	Új csoportot hoz létre a groupname és description paraméterekkel

```
def test_group_creation(self):
        """Test group creation automatically adds creator as leader"""
        self.client.force_authenticate(user=self.user1)

        # Create group
        group_data = {
            'groupname': 'Test Project Group',
            'description': 'A descriptive group'
        }
        res = self.client.post(self.groups_url, group_data)
        self.assertEqual(res.status_code, status.HTTP_201_CREATED)
        group_id = res.data['id']

        # Ensure creator is leader
        leader_member = GroupMember.objects.get(group_id=group_id,
user=self.user1)
        self.assertEqual(leader_member.role, 'leader')
```

1.2.2 Csoporttag hozzáadása

Teszteset ID	TC-GRP-02
API végpont	/api/group-members/
HTTP metódus	POST
Elvárt működés / Ellenőrzés	Sikeresen hozzárendel egy felhasználót (user) egy csoporthoz (group) a megadott szerepkörrel (pl. „reader”).

```
def test_member_addition(self):
        """Test adding another member to an existing group"""
        self.client.force_authenticate(user=self.user1)

        group = Group.objects.create(
            groupname="Test Add Member Group",
            created_by_userid=self.user1
        )
```

```
        GroupMember.objects.create(group=group, user=self.user1,
role='leader')

        # Add user2 to the group as an operator
        member_data = {
            'group': group.id,
            'user': self.user2.id,
            'role': 'operator'
        }
        add_res = self.client.post(self.group_members_url, member_data)
        self.assertEqual(add_res.status_code, status.HTTP_201_CREATED)

        member_count = GroupMember.objects.filter(group_id=group.id).count()
        self.assertEqual(member_count, 2)
```

1.2.3 Csoport törlése

Teszteset ID	TC-GRP-03
API végpont	/api/groups/{id}
HTTP metódus	DELETE
Elvárt működés / Ellenőrzés	Az id adatot megadva az id-hez rendelt csoportot soft delete eljárás alapján törli (inaktívvá állítja)

```
def test_group_soft_deletion(self):
        """Test that deleting a group marks it as inactive"""
        self.client.force_authenticate(user=self.user1)

        group = Group.objects.create(
            groupname="Group to delete",
            created_by_userid=self.user1
        )
        GroupMember.objects.create(group=group, user=self.user1,
role='leader')

        res = self.client.delete(f"{self.groups_url}{group.id}/")
        self.assertEqual(res.status_code, status.HTTP_204_NO_CONTENT)

        # Retrieve from DB to check soft delete
        group.refresh_from_db()
        self.assertFalse(group.active)
```

1.3 Feladatkezelés

1.3.1 Személyes feladat hozzáadása

Teszteset ID	TC-TASK-01
API végpont	/api/tasks/
HTTP metódus	POST
Elvárt működés / Ellenőrzés	A rendszer létrehoz egy feladatot, ha a title, description, priority, status, created_by_userid és assigned_to_userid

```
def test_personal_task_creation(self):
        """Test creating a personal task"""
        self.client.force_authenticate(user=self.user1)

        personal_task_data = {
            'title': 'Buy groceries',
            'description': 'Milk, Eggs, Bread',
            'priority': 'medium',
            'status': 'todo',
            'created_by_userid': self.user1.id,
            'assigned_to_userid': self.user1.id,
        }
        p_res = self.client.post(self.tasks_url, personal_task_data)
        self.assertEqual(p_res.status_code, status.HTTP_201_CREATED)
        self.assertIsNone(p_res.data.get('group_detail'))
```

1.3.2 Csoportos feladat létrehozása

Teszteset ID	TC-TASK-02
API végpont	/api/tasks/
HTTP metódus	POST
Elvárt működés / Ellenőrzés	Sikeresen létrehoz egy adott csoporthoz rendelt feladatot, ha a group azonosító szerepel a kérés törzsében.

```
def test_group_task_creation(self):
        """Test creating a group task"""
        self.client.force_authenticate(user=self.user1)

        # Group Task setup (Creating group first)
        group = Group.objects.create(groupname='Work', description='Work
tasks', created_by_userid=self.user1)
        GroupMember.objects.create(group=group, user=self.user1,
role='leader')
```

```
        GroupMember.objects.create(group=group, user=self.user2,
role='operator')

        # Create Task for Group
        group_task_data = {
            'title': 'Finish Backend Tests',
            'description': 'Write comprehensive unit tests',
            'priority': 'high',
            'status': 'todo',
            'group': group.id,
            'created_by_userid': self.user1.id,
            'assigned_to_userid': self.user1.id,
        }
        g_res = self.client.post(self.tasks_url, group_task_data)
        self.assertEqual(g_res.status_code, status.HTTP_201_CREATED)

        # Verify assignment
        task = Task.objects.get(id=g_res.data['id'])
        self.assertEqual(task.assigned_to_userid.id, self.user1.id)
```

1.3.3 Feladat módosítása

Teszteset ID	TC-TASK-03
API végpont	/api/tasks/{id}/
HTTP metódus	PATCH
Elvárt működés / Ellenőrzés	Frissített mezők küldése (pl. {„title”: „Frissített cím”}) módosítja a feladatot, és a változtatás mentésre kerül az adatbázisban.

```
def test_task_edit(self):
        """Test editing a task's properties"""
        self.client.force_authenticate(user=self.user1)

        # Create a task to edit
        task = Task.objects.create(
            title='Initial Title',
            description='Initial Description',
            priority='medium',
            status='todo',
            created_by_userid=self.user1,
        )

        edit_data = {
            'title': 'Updated Title',
            'status': 'in_progress'
        }
```

```
        res = self.client.patch(f"{self.tasks_url}{task.id}/", edit_data)

        self.assertEqual(res.status_code, status.HTTP_200_OK)

        task.refresh_from_db()
        self.assertEqual(task.title, 'Updated Title')
        self.assertEqual(task.status, 'in_progress')
        self.assertEqual(task.description, 'Initial Description')
```

1.3.4 Feladatok szűrése

Teszteset ID	TC-TASK-05
API végpont	/api/tasks/
HTTP metódus	GET
Elvárt működés / Ellenőrzés	A lekérdezési paraméterek használata sikeresen kiszűri és visszaadja a megadott paramétereknek megfelelő feladatokat. Ez támogatja a Frontend szűrési követelményeit.

```
def test_filter_tasks(self):
        """Test filtering tasks by status, group, and group__isnull"""
        self.client.force_authenticate(user=self.user1)

        # Setup specific groups and tasks
        group1 = Group.objects.create(groupname="Filter Group",
created_by_userid=self.user1)
        GroupMember.objects.create(group=group1, user=self.user1,
role='leader')

        task_todo_personal = Task.objects.create(title="T1",
created_by_userid=self.user1, status='todo')
        task_inprog_personal = Task.objects.create(title="T2",
created_by_userid=self.user1, status='in_progress')
        task_done_group = Task.objects.create(title="T3",
created_by_userid=self.user1, status='done', group=group1)

        # Filter by status
        res_status = self.client.get(f"{self.tasks_url}?status=todo")
        self.assertEqual(res_status.status_code, status.HTTP_200_OK)
        ids = [t['id'] for t in res_status.data]
        self.assertIn(task_todo_personal.id, ids)
        self.assertNotIn(task_inprog_personal.id, ids)

        # Filter by group
```

```
        res_group = self.client.get(f"{self.tasks_url}?group={group1.id}")
        self.assertEqual(res_group.status_code, status.HTTP_200_OK)
        # Verify only task_done_group is in the result (among the ones we
created)
        group_ids = [t['id'] for t in res_group.data]
        self.assertIn(task_done_group.id, group_ids)
        self.assertNotIn(task_todo_personal.id, group_ids)

        # Filter by group IS NULL (Personal Tasks)
        res_null = self.client.get(f"{self.tasks_url}?group__isnull=true")
        self.assertEqual(res_null.status_code, status.HTTP_200_OK)
        null_ids = [t['id'] for t in res_null.data]
        self.assertIn(task_todo_personal.id, null_ids)
        self.assertNotIn(task_done_group.id, null_ids)
```

1.3.5 Feladat törlése

Teszteset ID	TC-TASK-04
API végpont	/api/tasks/{id}/
HTTP metódus	DELETE
Elvárt működés / Ellenőrzés	A feladatot inaktívvá teszi

```
def test_task_soft_deletion(self):
        """Test that deleting a task marks it as inactive instead of
removing it from DB"""
        self.client.force_authenticate(user=self.user1)

        task = Task.objects.create(
            title="Task to delete",
            created_by_userid=self.user1,
            priority='high',
            status='todo'
        )

        res = self.client.delete(f"{self.tasks_url}{task.id}/")
        self.assertEqual(res.status_code, status.HTTP_204_NO_CONTENT)

        # Retrieve from DB to check soft delete
        task.refresh_from_db()
        self.assertFalse(task.active)
```

1.4 Lekérések

1.4.1 Felhasználók lekérése

Teszteset ID	TC-GET-01
API végpont	/api/users/
HTTP metódus	GET
Elvárt működés / Ellenőrzés	Végponthoz hozzáférve minden felhasználót visszaad

```
def test_list_users(self):
        """Test retrieving all active users"""
        self.client.force_authenticate(user=self.user1)
        res_list = self.client.get(self.users_url)
        self.assertEqual(res_list.status_code, status.HTTP_200_OK)
        self.assertGreaterEqual(len(res_list.data), 2)
```

1.4.2 Adott felhasználó lekérése

Teszteset ID	TC-GET-02
API végpont	/api/users/{id}
HTTP metódus	GET
Elvárt működés / Ellenőrzés	Az id adatot megadva az id-hez rendelt felhasználót visszaadja

```
def test_retrieve_user(self):
        """Test retrieving a specific user by ID"""
        self.client.force_authenticate(user=self.user1)
        res_detail = self.client.get(f"{self.users_url}{self.user2.id}/")
        self.assertEqual(res_detail.status_code, status.HTTP_200_OK)
        self.assertEqual(res_detail.data['id'], self.user2.id)
        self.assertEqual(res_detail.data['username'], self.user2.username)
```

1.4.3 Csoportok lekérése

Teszteset ID	TC-GET-03
API végpont	/api/groups/
HTTP metódus	GET
Elvárt működés / Ellenőrzés	Végponthoz hozzáférve minden csoportot visszaad

```
def test_list_groups(self):
        """Test retrieving all groups"""
        self.client.force_authenticate(user=self.user1)
        Group.objects.create(groupname="Global Group 1",
created_by_userid=self.user1)
```

```
        Group.objects.create(groupname="Global Group 2",
created_by_userid=self.user1)

        res_list = self.client.get(self.groups_url)
        self.assertEqual(res_list.status_code, status.HTTP_200_OK)
        self.assertGreaterEqual(len(res_list.data), 2)
```

1.4.4 Adott csoport lekérése

Teszteset ID	TC-GET-04
API végpont	/api/groups/{id}
HTTP metódus	GET
Elvárt működés / Ellenőrzés	Az id adatot megadva az id-hez rendelt csoportot visszaadja

```
def test_retrieve_group(self):
        """Test retrieving a specific group by ID"""
        self.client.force_authenticate(user=self.user1)
        group1 = Group.objects.create(groupname="Specific Group 1",
created_by_userid=self.user1)

        res_detail = self.client.get(f"{self.groups_url}{group1.id}/")
        self.assertEqual(res_detail.status_code, status.HTTP_200_OK)
        self.assertEqual(res_detail.data['id'], group1.id)
        self.assertEqual(res_detail.data['groupname'], group1.groupname)
```

1.4.5 Feladatok lekérése

Teszteset ID	TC-GET-05
API végpont	/api/tasks/
HTTP metódus	GET
Elvárt működés / Ellenőrzés	Végponthoz hozzáférve minden feladatot visszaad

```
def test_list_tasks(self):
        """Test retrieving all tasks"""
        self.client.force_authenticate(user=self.user1)
        Task.objects.create(title="Task A", created_by_userid=self.user1,
status='todo')
        Task.objects.create(title="Task B", created_by_userid=self.user1,
status='in_progress')

        res_list = self.client.get(self.tasks_url)
        self.assertEqual(res_list.status_code, status.HTTP_200_OK)
```

1.4.6 Adott feladat lekérése

Teszteset ID	TC-GET-06
API végpont	/api/tasks/{id}
HTTP metódus	GET
Elvárt működés / Ellenőrzés	Az id adatot megadva az id-hez rendelt feladatot visszaadja

```
def test_retrieve_task(self):
        """Test retrieving a specific task by ID"""
        self.client.force_authenticate(user=self.user1)
        task1 = Task.objects.create(title="Task A",
created_by_userid=self.user1, status='todo')

        res_detail = self.client.get(f"{self.tasks_url}{task1.id}/")
        self.assertEqual(res_detail.status_code, status.HTTP_200_OK)
        self.assertEqual(res_detail.data['id'], task1.id)
        self.assertEqual(res_detail.data['title'], task1.title)
```